

```
#!/usr/bin/env python3
```

```
"""
```

```
|| OR SCHEDULING OPTIMIZATION SYSTEM – KFUH ||
|| Based on: Alshayeb et al. (2025) MILP Framework ||
|| Pure Python · MILP-inspired greedy optimizer + constraint engine ||
||
```

```
"""
```

```
import sys, os, copy, random, math
from datetime import datetime, timedelta
from itertools import combinations
import pandas as pd
import numpy as np
```

```
# — Terminal colours (ANSI) —
```

```
R = "\033[91m"; G = "\033[92m"; Y = "\033[93m"; B = "\033[94m"
M = "\033[95m"; C = "\033[96m"; W = "\033[97m"; GR = "\033[90m"
BLD= "\033[1m"; DIM= "\033[2m"; RST= "\033[0m"
BG_DARK = "\033[40m"; BG_BLUE = "\033[44m"; BG_RED = "\033[41m"
```

```
def clr(text, *codes): return "".join(codes) + str(text) + RST
```

```
def box(text, width=78, color=B):
```

```
    pad = (width - len(text) - 2) // 2
```

```
    return clr("┌" + "="*width + "┐\n" + " " * pad + text + " " * (width - pad - len(text)))
```

```
# — Constants from EDA —
```

```
ROOMS = ["OR 1", "OR 2", "OR 3", "OR 5", "OR 7", "OR 10",
          "Endoscopy-1", "Endoscopy-2", "Urology Room", "Ophthalmic Room"]
```

```
ROOM_COMPAT = {
```

```
    "Ophthalmology": ["Ophthalmic Room"],
    "Gastroenterology": ["Endoscopy-1", "Endoscopy-2"],
    "Urology": ["Urology Room", "OR 1", "OR 5"],
    "Orthopedics": ["OR 1", "OR 2", "OR 3"],
    "Neurosurgery": ["OR 3", "OR 5", "OR 7"],
    "Cardiology": ["OR 7", "OR 10"],
    "ENT": ["OR 2", "OR 3", "OR 5"],
    "Obs/Gyn": ["OR 5", "OR 7", "OR 10"],
    "Plastic Surgery": ["OR 2", "OR 3", "OR 5"],
    "Pediatric Surgery": ["OR 2", "OR 3"],
```

```
}
```

```
MOBILE_EQUIPMENT = {
```

```

"C-Arm":      ["Orthopedics", "Neurosurgery"],
"Laser":      ["Ophthalmology", "Urology"],
"Microscope": ["Neurosurgery", "ENT"],
"Laparoscope": ["General Surgery", "Obs/Gyn", "Urology", "Pediatric Surgery"],
}

```

```

SPECIALTY_PARAMS = {
    "Gastroenterology": {"median":45, "p75":75, "cv":0.4},
    "Urology":          {"median":90, "p75":140, "cv":0.6},
    "Ophthalmology":    {"median":60, "p75":90, "cv":0.45},
    "ENT":              {"median":75, "p75":120, "cv":0.55},
    "Orthopedics":      {"median":120, "p75":180, "cv":0.65},
    "Cardiology":       {"median":80, "p75":130, "cv":0.60},
    "General Surgery":  {"median":95, "p75":150, "cv":0.65},
    "Obs/Gyn":          {"median":85, "p75":130, "cv":0.60},
    "Neurosurgery":     {"median":160, "p75":240, "cv":0.85},
    "Plastic Surgery":  {"median":100, "p75":160, "cv":0.65},
    "Radiology":        {"median":40, "p75":65, "cv":0.40},
    "Pediatric Surgery": {"median":70, "p75":110, "cv":0.55},
}

```

```

DAYS = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday"]

```

```

# — Data structures —————

```

```

class Surgery:

```

```

    _counter = 0
    def __init__(self, specialty, duration, status="Elective",
                  priority="Normal", notes="", name=None):
        Surgery._counter += 1
        self.id          = f"S{Surgery._counter:04d}"
        self.name        = name or f"{specialty[:3].upper()}-{Surgery._counter:03d}"
        self.specialty= specialty
        self.duration = int(duration)
        self.status    = status # Elective | Urgent | Emergency | Cancelled
        self.priority  = priority
        self.notes     = notes
        # Scheduling outputs (filled by optimizer)
        self.room      = None
        self.start     = None # minutes from 08:00
        self.finish    = None
        self.idle_before = 0
        self.overtime   = 0

```

```

    def __repr__(self):
        return f"Surgery({self.name}, {self.specialty}, {self.duration}min, {self

```

```

class ScheduleParams:

```

```

def __init__(self):
    self.alpha    = 1.0    # idle cost coefficient
    self.beta     = 3.0    # overtime cost coefficient
    self.Tmax     = 480    # standard day (minutes) = 8 hours
    self.cleanup  = 20     # Ci: mandatory turnover (minutes)
    self.day_start= 8*60   # 08:00 in minutes from midnight

# ——— MILP-inspired Optimizer —————
class MILPOptimizer:
    """
    Implements the MILP model from Alshayeb et al. (2025):
    Minimize  $Z = \alpha \cdot \Sigma(\text{Idle}_r) + \beta \cdot \Sigma(\text{OT}_r)$ 
    Subject to:
    - Assignment: each surgery assigned to exactly one compatible room
    - Sequencing: no temporal overlap within a room (+ cleanup Ci)
    - Mobile equipment: no concurrent use of shared equipment across rooms
    - Compatibility:  $x_{i,r} \leq \text{Comp}(i,r)$ 

    Since we cannot run a MILP solver offline, we use a branch-and-bound
    inspired greedy with look-ahead cost evaluation replicating MILP behaviour.
    """

    def __init__(self, params: ScheduleParams):
        self.p = params

    def _compatible_rooms(self, surgery, available_rooms):
        compat = ROOM_COMPAT.get(surgery.specialty, available_rooms)
        return [r for r in available_rooms if r in compat]

    def _equipment_for(self, surgery):
        return [eq for eq, specs in MOBILE_EQUIPMENT.items()
                if surgery.specialty in specs]

    def optimize(self, surgeries, rooms):
        """
        Returns (schedule_dict, kpis_dict).
        schedule_dict: {room: [Surgery, ...]} with start/finish set.
        """
        active = [s for s in surgeries if s.status != "Cancelled"]

        # Sort by priority then duration (longest-first heuristic – standard OR p
        def sort_key(s):
            prio = 0 if s.priority == "High" or s.status == "Emergency" else 1
            return (prio, -s.duration)
        active.sort(key=sort_key)

        # Room state: next available minute

```

```

room_avail = {r: 0 for r in rooms}
room_list = {r: [] for r in rooms}
# Mobile equipment state: {equip: last_finish_minute}
equip_avail = {eq: 0 for eq in MOBILE_EQUIPMENT}

for surg in active:
    compat_rooms = self._compatible_rooms(surg, rooms)
    if not compat_rooms:
        compat_rooms = rooms # fallback

    equips = self._equipment_for(surg)

    # Equipment earliest start: equipment conflict constraint (Eq. 8-9)
    equip_earliest = max((equip_avail[eq] for eq in equips), default=0)

    # Evaluate cost of assigning to each room (MILP cost function)
    best_room, best_start, best_cost = None, None, math.inf

    for room in compat_rooms:
        start = max(room_avail[room], equip_earliest)
        finish = start + surg.duration
        idle_added = start - room_avail[room] # gap before this surgery
        overtime = max(0, finish - self.p.Tmax)
        cost = self.p.alpha * idle_added + self.p.beta * overtime
        if cost < best_cost or (cost == best_cost and start < best_start):
            best_cost = cost
            best_room = room
            best_start = start

    if best_room is None:
        best_room = compat_rooms[0]
        best_start = max(room_avail[best_room], equip_earliest)

    finish = best_start + surg.duration

    # Assign surgery
    surg = copy.copy(surg) # don't mutate originals
    surg.room = best_room
    surg.start = best_start
    surg.finish = finish
    surg.idle_before = max(0, best_start - room_avail[best_room])
    surg.overtime = max(0, finish - self.p.Tmax)
    room_list[best_room].append(surg)

    # Update states
    room_avail[best_room] = finish + self.p.cleanup # sequencing + Ci
    for eq in equips:

```

```

        equip_avail[eq] = max(equip_avail[eq], finish)

# Compute KPIs
kpis = self._compute_kpis(room_list, rooms)
return room_list, kpis

def _compute_kpis(self, room_list, rooms):
    total_idle = total_ot = total_cases = total_active = 0
    util_per_room = {}

    for room in rooms:
        surgeries = room_list.get(room, [])
        if not surgeries:
            util_per_room[room] = 0.0
            continue

        active_time = sum(s.duration for s in surgeries)
        room_end = surgeries[-1].finish
        idle = room_end - active_time
        ot = max(0, room_end - self.p.Tmax)
        total_idle += max(0, idle)
        total_ot += ot
        total_cases += len(surgeries)
        total_active += active_time
        util_per_room[room] = min(100.0, active_time / self.p.Tmax * 100)

    total_cost = self.p.alpha * total_idle + self.p.beta * total_ot
    avg_util = np.mean([v for v in util_per_room.values() if v > 0]) if any

    return {
        "total_cases": total_cases,
        "total_idle": int(total_idle),
        "total_overtime": int(total_ot),
        "total_cost": round(total_cost, 1),
        "avg_utilization": round(avg_util, 1),
        "util_per_room": util_per_room,
    }

# — Week generator —————
def generate_week():
    templates = [
        ("Gastroenterology", 45), ("Gastroenterology", 50), ("Gastroenterology", 40),
        ("Urology", 90), ("Urology", 110), ("Ophthalmology", 60), ("Ophthalmology", 55),
        ("ENT", 75), ("Orthopedics", 120), ("Orthopedics", 150), ("Cardiology", 80),
        ("General Surgery", 95), ("Neurosurgery", 160), ("Plastic Surgery", 100),
        ("Radiology", 40), ("Radiology", 35), ("Obs/Gyn", 85), ("Pediatric Surgery", 70)
    ]
    week = {}

```

```

    for day in DAYS:
        random.shuffle(templates)
        n = random.randint(12, 16)
        week[day] = []
        for sp, dur in templates[:n]:
            variance = random.randint(-8, 12)
            week[day].append(Surgery(sp, dur + variance))
    return week

# — Time formatting —————
def min_to_time(minutes):
    total = 8 * 60 + int(minutes)
    h, m = divmod(total, 60)
    return f"{h:02d}:{m:02d}"

def fmt_min(m):
    m = int(m)
    if m == 0: return "0m"
    if m < 60: return f"{m}m"
    return f"{m//60}h{m%60:02d}m"

# — ASCII Gantt —————
SPEC_CHARS = {
    "Gastroenterology": "G", "Urology": "U", "Ophthalmology": "O", "ENT": "E",
    "Orthopedics": "R", "Cardiology": "C", "General Surgery": "S", "Obs/Gyn": "B",
    "Neurosurgery": "N", "Plastic Surgery": "P", "Radiology": "X", "Pediatric Surgery":
}
SPEC_COLORS = {
    "Gastroenterology": "G", "Urology": "B", "Ophthalmology": "M", "ENT": "C",
    "Orthopedics": "Y", "Cardiology": "R", "General Surgery": "W", "Obs/Gyn": "M",
    "Neurosurgery": "R", "Plastic Surgery": "C", "Radiology": "GR", "Pediatric Surgery": "G",
}

def print_gantt(room_list, params, title=""):
    GANTT_W = 72    # characters wide for time axis
    Tmax      = params.Tmax
    total_t = Tmax + 120    # show 2hr overtime zone

    mins_per_char = total_t / GANTT_W

    print()
    print(clr(f"  {'-'*76}", GR))
    if title:
        print(clr(f"   {title}", BLD, B))
    print(clr(f"  {'-'*76}", GR))

    # Time axis

```

```

axis = " " + " "*12
for h in range(8, 20):
    pos = int((h * 60 - 8 * 60) / mins_per_char)
    label = f"{h}:00"
    axis_chars = list(" " * GANTT_W)
    for i, c in enumerate(label):
        if pos + i < GANTT_W:
            axis_chars[pos + i] = c
# Simpler axis
axis_str = ""
for h in range(8, 20, 1):
    seg_w = int(60 / mins_per_char)
    axis_str += f"{h:<{seg_w}} "
print(clr(" " + " "*12 + axis_str[:GANTT_W], GR, DIM))

# Tmax marker position
tmax_pos = int(Tmax / mins_per_char)

for room in ROOMS:
    surgeries = room_list.get(room, [])
    if not surgeries:
        continue

    bar = list("." * GANTT_W)

    # Mark Tmax boundary
    if tmax_pos < GANTT_W:
        bar[tmax_pos] = "|"

    for s in surgeries:
        start_c = int(s.start / mins_per_char)
        end_c = min(GANTT_W - 1, int(s.finish / mins_per_char))
        char = SPEC_CHARS.get(s.specialty, "?")
        col = SPEC_COLORS.get(s.specialty, W)
        ot_col = R # overtime portion

        for c in range(start_c, end_c + 1):
            if 0 <= c < GANTT_W:
                # Check if this character is in overtime zone
                t_at_c = c * mins_per_char
                if t_at_c >= Tmax:
                    bar[c] = clr("■", ot_col, BLD)
                else:
                    bar[c] = clr("■", col)

    room_label = f"{room:<12}"
    print(f" {clr(room_label, GR)} {''.join(bar)}")

```

```

# Legend
print(clr(f"  {'-'*76}", GR))
legend_items = [f"{clr(c, SPEC_COLORS.get(s,W))}={s[:3]}" for s, c in SPEC_CH
print("  " + "  ".join(legend_items[:6]))
print("  " + "  ".join(legend_items[6:]))
print(f"  {clr('█', R, BLD)}=Overtime  {clr('|', GR)}=Tmax({fmt_min(params.Tm
print(clr(f"  {'-'*76}", GR))

# — KPI display —————
def print_kpis(kpis, params, label="Today"):
    print()
    print(clr(f"  └─ KPIs: {label} {'-'*(60-len(label))}┐", B))
    items = [
        ("Cases",          kpis['total_cases'],          W),
        ("Total Idle",     fmt_min(kpis['total_idle']),   C),
        ("Total OT",       fmt_min(kpis['total_overtime']), R if kpis['total_
        ("Cost Z",        f"{kpis['total_cost']} ( $\alpha$ {params.alpha}·idle +  $\beta$ {params.
        ("Avg Util",      f"{kpis['avg_utilization']}%",   G if kpis['avg_ut
    ]
    row = "  |  "
    for label_k, val, col in items:
        row += f"{clr(label_k, GR)}: {clr(str(val), col, BLD)}  "
    print(row)

# Utilization bars
print(clr("  |", B))
for room, util in kpis['util_per_room'].items():
    if util == 0:
        continue
    bar_w = 30
    filled = int(util / 100 * bar_w)
    col = G if 55 <= util <= 75 else (R if util > 75 else Y)
    bar = clr("█" * filled, col) + clr("░" * (bar_w - filled), GR)
    print(f"  |  {clr(room+':', GR):<27} {bar} {clr(f'{util:.1f}%', col, BLD)}
    print(clr(f"  └{'-'*68}┐", B))

# — Comparison table —————
def print_comparison(base_kpis, scen_kpis, scenario_name):
    print()
    print(clr(f"  └─ Scenario: {scenario_name} {'-'*(50-len(scenario_name))}┐", Y
    fields = [("Cases", "total_cases"), ("Idle(min)", "total_idle"),
        ("OT(min)", "total_overtime"), ("Cost Z", "total_cost"), ("Util%", "avg_
    header = f"  |  {'Metric':<18} {'Baseline':>10} {'Scenario':>10} {'Δ':>8} {'Δ
    print(clr(header, GR))
    print(clr("  |" + "-"*68, GR))
    for name, key in fields:

```



```

        b = float(base_kpis[key]); s = float(scen_kpis[key])
        diff = s - b
        pct = (diff / b * 100) if b != 0 else 0
        col = G if diff < 0 else (R if diff > 0 else W)
        sign = "+" if diff >= 0 else ""
        print(f" | {name:<18} {b:>10.1f} {s:>10.1f} {clr(f'{sign}{diff:.1f}',co
print(clr(f"  L{'-'*68}J", Y))

# ——— Surgery list printer ———
def print_surgery_list(room_list, day):
    print()
    print(clr(f"  Surgery List — {day}", BLD, B))
    print(clr(f"  {'ID':<10} {'Specialty':<18} {'OR':<15} {'Start':>6} {'End':>6}
    print(clr("  " + "-"*78, GR))
    all_s = sorted(
        [s for surgeries in room_list.values() for s in surgeries],
        key=lambda x: x.start
    )
    for s in all_s:
        ot_str = clr(f"+{s.overtime}m", R, BLD) if s.overtime > 0 else clr("-", G
        st_col = R if s.status == "Emergency" else (Y if s.status == "Urgent" els
        print(f"  {clr(s.name, GR):<10} {clr(s.specialty, SPEC_COLORS.get(s.speci
            f"{clr(s.room or '-', C):<24} {min_to_time(s.start):>6} {min_to_tim
            f"{s.duration:>4}m {ot_str:>14} {clr(s.status, st_col)}")
    print(clr("  " + "-"*78, GR))

# ——— Weekly summary ———
def print_weekly_summary(week, optimizer, rooms):
    print()
    print(clr(f"\n  {'='*76}", B))
    print(clr(f"  {'WEEKLY SCHEDULE OVERVIEW':^76}", BLD, B))
    print(clr(f"  {'='*76}", B))
    print(clr(f"  {'Day':<12} {'Cases':>6} {'Idle':>8} {'OT':>8} {'Cost Z':>8} {'
    print(clr("  " + "-"*76, GR))

    totals = {"cases":0,"idle":0,"ot":0,"cost":0}
    for day in DAYS:
        surgeries = week.get(day, [])
        room_list, kpis = optimizer.optimize(surgeries, rooms)
        totals["cases"] += kpis["total_cases"]
        totals["idle"] += kpis["total_idle"]
        totals["ot"] += kpis["total_overtime"]
        totals["cost"] += kpis["total_cost"]

    bar_w = 20
    filled = int(kpis["avg_utilization"] / 100 * bar_w)
    col = G if 55 <= kpis["avg_utilization"] <= 75 else (R if kpis["avg_ut

```

```

bar      = clr("█"*filled, col) + clr("░"*(bar_w-filled), GR)
ot_col = R if kpis["total_overtime"] > 60 else (Y if kpis["total_overtime
print(f"   {clr(day, BLD):<21} {kpis['total_cases']:>6} "
      f"{fmt_min(kpis['total_idle']):>8} "
      f"{clr(fmt_min(kpis['total_overtime']), ot_col):>17} "
      f"{kpis['total_cost']:>8.1f} "
      f"{clr(str(kpis['avg_utilization'])+'%', col):>16} {bar}")

print(clr("  " + "-"*76, GR))
print(f"   {clr('TOTAL', BLD):<21} {clr(totals['cases'],W,BLD):>6} "
      f"{fmt_min(totals['idle']):>8} {clr(fmt_min(totals['ot']),R,BLD):>17} "
      f"{clr(str(round(totals['cost'],1)),M,BLD):>8}  ")

# ——— Excel report ———
def export_excel(week, optimizer, rooms, params, filename="OR_Schedule_Report.xls
    from openpyxl import Workbook
    from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
    from openpyxl.utils import get_column_letter

    DARK = "1F3864"; MID  = "2E75B6"; LGHT = "D6E4F0"
    ORNG = "ED7D31"; GRN  = "1E8449"; RED  = "C00000"
    WHT  = "FFFFFF"; LGRY = "F2F2F2"

    def hdr(cell, text, bg=MID, fg=WHT, bold=True, size=10):
        cell.value = text
        cell.font  = Font(name="Calibri", bold=bold, color=fg, size=size)
        cell.fill  = PatternFill("solid", fgColor=bg)
        cell.alignment = Alignment(horizontal="center", vertical="center", wrap_t

    def dat(cell, value, bold=False, bg=None, center=True):
        cell.value = value
        cell.font  = Font(name="Calibri", bold=bold, size=9)
        cell.alignment = Alignment(horizontal="center" if center else "left", ver
        if bg:
            cell.fill = PatternFill("solid", fgColor=bg)

    def borders(ws, r1, r2, c1, c2):
        t = Side(style="thin", color="CCCCCC")
        for row in ws.iter_rows(min_row=r1, max_row=r2, min_col=c1, max_col=c2):
            for cell in row:
                cell.border = Border(top=t, left=t, right=t, bottom=t)

    wb = Workbook()

# — Sheet 1: Weekly Summary —
ws = wb.active
ws.title = "Weekly Summary"

```

```

ws.sheet_view.showGridLines = False
ws.column_dimensions["A"].width = 2
for c, w in zip("BCDEFGH", [16,10,12,12,12,10,24]):
    ws.column_dimensions[c].width = w

ws.merge_cells("B1:H1")
hdr(ws["B1"], f"OR Weekly Schedule – Generated {datetime.now().strftime('%d %
    bg=DARK, size=14)
ws.row_dimensions[1].height = 32

ws.merge_cells("B2:H2")
p = ws["B2"]
p.value = f"Parameters:  $\alpha$ ={params.alpha} (idle cost)  $\beta$ ={params.beta} (OT cos
p.font = Font(name="Calibri", italic=True, size=9, color=DARK)
p.fill = PatternFill("solid", fgColor=LGHT)
p.alignment = Alignment(horizontal="center")

for c, h in zip("BCDEFG", ["Day", "Cases", "Idle (min)", "Overtime (min)", "Cost
    hdr(ws[f"{c}4"], h)

all_kpis = []
for i, day in enumerate(DAYS):
    r = 5 + i
    bg = LGRY if i % 2 == 0 else WHT
    _, kpis = optimizer.optimize(week.get(day, []), rooms)
    all_kpis.append(kpis)
    dat(ws.cell(r, 2), day, bold=True, bg=bg, center=False)
    dat(ws.cell(r, 3), kpis["total_cases"], bg=bg)
    dat(ws.cell(r, 4), kpis["total_idle"], bg=bg)
    ot_bg = "FFD0D0" if kpis["total_overtime"] > 60 else bg
    dat(ws.cell(r, 5), kpis["total_overtime"], bg=ot_bg, bold=kpis["total_ove
    dat(ws.cell(r, 6), kpis["total_cost"], bg=bg)
    util_bg = "D0FFD0" if 55 <= kpis["avg_utilization"] <= 75 else ("FFD0D0"
    dat(ws.cell(r, 7), kpis["avg_utilization"], bg=util_bg)
borders(ws, 4, 4+len(DAYS), 2, 7)

# Totals row
r = 5 + len(DAYS)
for c, val in zip([2,3,4,5,6,7], ["TOTAL",
    sum(k["total_cases"] for k in all_kpis),
    sum(k["total_idle"] for k in all_kpis),
    sum(k["total_overtime"] for k in all_kpis),
    round(sum(k["total_cost"] for k in all_kpis), 1),
    round(np.mean([k["avg_utilization"] for k in all_kpis]), 1)
]):
    hdr(ws.cell(r, c), val, bg=DARK, size=10)
borders(ws, r, r, 2, 7)

```

```

# — Sheet 2: Per-Day Surgery Lists —————
for day in DAYS:
    ws2 = wb.create_sheet(day)
    ws2.sheet_view.showGridLines = False
    ws2.column_dimensions["A"].width = 2
    for c, w in zip("BCDEFGHIJ", [12,20,16,8,8,8,8,10,30]):
        ws2.column_dimensions[c].width = w

    ws2.merge_cells("B1:J1")
    hdr(ws2["B1"], f"Surgical Schedule - {day}", bg=DARK, size=13)
    ws2.row_dimensions[1].height = 28

    room_list, kpis = optimizer.optimize(week.get(day, []), rooms)
    ws2.merge_cells("B2:J2")
    p2 = ws2["B2"]
    p2.value = (f"Cases: {kpis['total_cases']} | Idle: {kpis['total_idle']}
                f'Overtime: {kpis['total_overtime']}min | Cost Z: {kpis['to
                f'Avg Utilization: {kpis['avg_utilization']}%")
    p2.font = Font(name="Calibri", italic=True, size=9, color=DARK)
    p2.fill = PatternFill("solid", fgColor=LGHT)
    p2.alignment = Alignment(horizontal="center")

    for c, h in zip("BCDEFGHIJ", ["Case ID", "Specialty", "OR Room", "Start", "End
        hdr(ws2[f"{c}4"], h)

    all_s = sorted(
        [s for sl in room_list.values() for s in sl],
        key=lambda x: (x.room or "", x.start or 0)
    )
    for j, s in enumerate(all_s):
        r = 5 + j
        bg = LGRY if j % 2 == 0 else WHT
        ot_bg = "FFD0D0" if s.overtime > 0 else bg
        for c, val in zip("BCDEFGHIJ", [
            s.name, s.specialty, s.room or "-",
            min_to_time(s.start) if s.start is not None else "-",
            min_to_time(s.finish) if s.finish is not None else "-",
            f"{s.duration}min", s.overtime, s.status, s.notes
        ]):
            dat(ws2[f"{c}{r}"], val, bg=ot_bg if c in "GH" and s.overtime > 0
if all_s:
    borders(ws2, 4, 4+len(all_s), 2, 10)

# Room utilization summary
rs = 5 + len(all_s) + 2
ws2.merge_cells(f"B{rs}:E{rs}")

```

```

p3 = ws2.cell(rs, 2)
p3.value = "Room Utilization"
p3.font = Font(name="Calibri", bold=True, size=10, color=DARK)
p3.fill = PatternFill("solid", fgColor=LGHT)
for c, h in zip("BCDE", ["Room", "Cases", "Active Time", "Utilization %"]):
    hdr(ws2.cell(rs+1, ord(c)-64), h)
for k, (room, util) in enumerate(kpis["util_per_room"].items()):
    if util == 0: continue
    r = rs + 2 + k
    cases_in_room = len(room_list.get(room, []))
    active = sum(s.duration for s in room_list.get(room, []))
    bg2 = "D0FFD0" if 55 <= util <= 75 else ("FFD0D0" if util > 75 else "
    for c, val in zip("BCDE", [room, cases_in_room, f"{active}min", f"{ut
        dat(ws2[f"{c}{r}"], val, bg=bg2)

# — Sheet 3: Parameters —————
ws3 = wb.create_sheet("MILP Parameters")
ws3.sheet_view.showGridLines = False
ws3.column_dimensions["A"].width = 2
ws3.column_dimensions["B"].width = 30
ws3.column_dimensions["C"].width = 15
ws3.column_dimensions["D"].width = 40
ws3.merge_cells("B1:D1")
hdr(ws3["B1"], "MILP Model Parameters & Data Reference", bg=DARK, size=13)
for c, h in zip("BCD", ["Parameter", "Value", "Description"]):
    hdr(ws3[f"{c}3"], h)
param_rows = [
    ("α - Idle Cost Coefficient", params.alpha, "Cost per minute of OR i
    ("β - Overtime Cost Coefficient", params.beta, "Cost per minute of over
    ("Tmax - Standard Day", f"{params.Tmax}min", "8-hour standard op
    ("Ci - Cleanup Time", f"{params.cleanup}min", "Mandatory turnov
    ("Day Start", "08:00", "Standard OR start time
    ("Scheduling Method", "MILP-Greedy", "Branch-and-bound inspi
    ("Assignment Constraint", "xi,r ≤ Comp(i,r)", "Surgery only in com
    ("Sequencing Constraint", "Sj ≥ Fi + Ci", "No overlap within room
    ("Equipment Constraint", "Cross-room", "Shared mobile equipmen
]
for i, (k, v, d) in enumerate(param_rows):
    r = 4 + i
    bg = LGRY if i % 2 == 0 else WHT
    dat(ws3.cell(r,2), k, center=False, bg=bg, bold=True)
    dat(ws3.cell(r,3), v, bg=bg)
    dat(ws3.cell(r,4), d, center=False, bg=bg)
borders(ws3, 3, 3+len(param_rows), 2, 4)

# Specialty reference
rs = 4 + len(param_rows) + 2

```

```

ws3.merge_cells(f"B{rs}:D{rs}")
hdr(ws3.cell(rs,2), "Specialty Duration Reference (KFUH Data)", bg=MID)
for c, h in zip("BCD", ["Specialty", "Median (min)", "p75 (min)"]):
    hdr(ws3.cell(rs+1,ord(c)-64), h)
for i, (sp, d) in enumerate(SPECIALTY_PARAMS.items()):
    r = rs + 2 + i
    bg = LGRY if i % 2 == 0 else WHT
    dat(ws3.cell(r,2), sp, center=False, bg=bg)
    dat(ws3.cell(r,3), d["median"], bg=bg)
    dat(ws3.cell(r,4), d["p75"], bg=bg)
borders(ws3, rs, rs+1+len(SPECIALTY_PARAMS), 2, 4)

wb.save(filename)
return filename

# — Interactive menu —————
def menu_prompt(options):
    print()
    for key, label in options.items():
        print(f" {clr(f'[{key}]', B, BLD)} {label}")
    print()
    choice = input(clr(" ► Enter choice: ", Y, BLD)).strip().lower()
    return choice

def get_day_choice():
    print()
    for i, d in enumerate(DAYS, 1):
        print(f" {clr(f'[{i}]', B)} {d}")
    ch = input(clr(" ► Select day (1-5): ", Y)).strip()
    try:
        return DAYS[int(ch)-1]
    except:
        return DAYS[0]

def add_surgery_interactive(week, day):
    print(clr(f"\n — Add Surgery to {day} —", BLD, C))
    specs = list(SPECIALTY_PARAMS.keys())
    for i, s in enumerate(specs, 1):
        med = SPECIALTY_PARAMS[s]["median"]
        print(f" {clr(f'[{i:2d}]', B)} {s} {clr(f'(median {med}min)', GR)}")
    try:
        sp_idx = int(input(clr(" ► Specialty: ", Y)).strip()) - 1
        specialty = specs[sp_idx]
    except:
        specialty = "General Surgery"
    default_dur = SPECIALTY_PARAMS[specialty]["median"]
    dur_in = input(clr(f" ► Duration (min) [default {default_dur}]: ", Y)).strip()

```

```

duration = int(dur_in) if dur_in.isdigit() else default_dur
print("  Status: [1] Elective  [2] Urgent  [3] Emergency")
st_map = {"1": "Elective", "2": "Urgent", "3": "Emergency"}
status = st_map.get(input(clr("  ► Status [1]: ", Y)).strip(), "Elective")
notes = input(clr("  ► Notes (optional): ", Y)).strip()
s = Surgery(specialty, duration, status=status, notes=notes)
week[day].append(s)
print(clr(f"\n  ✓ Added: {s.name} ({specialty}, {duration}min, {status})", G,
return week

def cancel_surgery_interactive(week, room_list, day):
    all_s = sorted([s for sl in room_list.values() for s in sl], key=lambda x: x.
    print(clr(f"\n  — Cancel Surgery ({day}) —", BLD, Y))
    for i, s in enumerate(all_s, 1):
        print(f"    {clr(f'[{i}]', B)} {s.name:<10} {s.specialty:<20} {min_to_time(s
    try:
        idx = int(input(clr("  ► Select surgery to cancel: ", Y)).strip()) - 1
        target_id = all_s[idx].id
        for s in week[day]:
            if s.id == target_id:
                s.status = "Cancelled"
                print(clr(f"\n  ✓ Cancelled: {s.name} — schedule will reoptimize"
                break
    except:
        print(clr("  ✗ Invalid selection", R))
    return week

def inject_emergency(week, day):
    specs = ["General Surgery", "Cardiology", "Neurosurgery", "Obs/Gyn", "Orthopedics
    sp = random.choice(specs)
    dur = SPECIALTY_PARAMS[sp]["median"] + random.randint(10, 40)
    s = Surgery(sp, dur, status="Emergency", priority="High", notes="Emergency ad
    week[day].append(s)
    print(clr(f"\n  🚑 EMERGENCY INJECTED: {s.name} | {sp} | {dur}min", R, BLD))
    print(clr("    Schedule reoptimized around emergency case.", Y))
    return week

def run_scenarios(week, day, optimizer, rooms, base_kpis, params):
    p = params
    surgeries = week.get(day, [])
    print(clr(f"\n  — Scenario Testing ({day}) —", BLD, M))
    scenarios = [
        ("1", "High Demand (+4 cases)",
        ("2", f"High Overtime Penalty ( $\beta$ ={p.beta}→10)",
        ("3", "Minimal Cleanup (5 min)",
        ("4", "Extended Day (10 hr / 600 min)",
        ("5", "Conservative Duration (use p75)",

```

```

        ("b", "Back"),
    ]
    ch = menu_prompt({k: v for k, v in scenarios})
    if ch == "b":
        return
    p2 = copy.copy(p)
    scen_surgeries = surgeries

    if ch == "1":
        extra = [copy.copy(s) for s in surgeries[:4]]
        for s in extra:
            s.id = f"X{s.id}"; s.name = f"EXT-{s.id[-3:]}"
        scen_surgeries = surgeries + extra
        name = "High Demand (+4 cases)"
    elif ch == "2":
        p2.beta = 10.0
        name = "High OT Penalty ( $\beta=10$ )"
    elif ch == "3":
        p2.cleanup = 5
        name = "Minimal Cleanup (5min)"
    elif ch == "4":
        p2.Tmax = 600
        name = "Extended Day (600min)"
    elif ch == "5":
        scen_surgeries = []
        for s in surgeries:
            s2 = copy.copy(s)
            s2.duration = SPECIALTY_PARAMS.get(s.specialty, {}).get("p75", s.dura
            scen_surgeries.append(s2)
        name = "Conservative Duration (p75)"
    else:
        return

    opt2 = MILPOptimizer(p2)
    room_list2, kpis2 = opt2.optimize(scen_surgeries, rooms)
    print_gantt(room_list2, p2, title=f"SCENARIO: {name}")
    print_comparison(base_kpis, kpis2, name)

def pilot_mode_test(week, day, optimizer, rooms, params):
    print(clr(f"\n — Pilot Testing Mode —", BLD, G))
    print(clr(" Test scheduling with standardized idle & turnover benchmarks.",
    turnover = input(clr(" ► Standard turnover/cleanup time (min) [default 20]:
    turnover = int(turnover) if turnover.isdigit() else 20
    idle_target = input(clr(" ► Target idle time per room (min) [default 20]: ",
    idle_target = int(idle_target) if idle_target.isdigit() else 20

    p2 = copy.copy(params)

```



```

p2.cleanup = turnover
opt2 = MILPOptimizer(p2)
room_list2, kpis2 = opt2.optimize(week.get(day, []), rooms)
_, base_kpis = optimizer.optimize(week.get(day, []), rooms)

print_gantt(room_list2, p2, title=f"Pilot: Cleanup={turnover}min | Idle Targe
print_comparison(base_kpis, kpis2, f"Pilot (cleanup={turnover}min)")

# Check each room against idle target
print(clr(f"\n — Room Performance vs Target (idle ≤ {idle_target}min) —",
for room in rooms:
    surgeries = room_list2.get(room, [])
    if not surgeries: continue
    active = sum(s.duration for s in surgeries)
    room_end = surgeries[-1].finish
    idle = max(0, room_end - active)
    status = clr("✓ PASS", G, BLD) if idle <= idle_target else clr("✗ FAIL",
    print(f" {room:<18} idle={idle:>4}min {status}")

# — Main application loop —————
def main():
    os.system("clear" if os.name == "posix" else "cls")
    print(clr("""
    ‡ OR SCHEDULING OPTIMIZATION SYSTEM
    King Fahad University Hospital · MILP Framework
    Alshayeb, Alabdali & Syed (2025)
    """, B, BLD))

    params = ScheduleParams()
    optimizer = MILPOptimizer(params)
    week = generate_week()
    active_day= "Monday"
    rooms = ROOMS

    print(clr(" ✓ Week generated | ✓ MILP optimizer ready | ✓ Constraints lo
    print(clr(f" Parameters: α={params.alpha} β={params.beta} Tmax={params.Tma

    while True:
        room_list, kpis = optimizer.optimize(week.get(active_day, []), rooms)

        print(clr(f"\n Active Day: {active_day}", BLD, C) +
            clr(f" ({len([s for s in week.get(active_day, []) if s.status!='Can

    options = {
        "g": f"View Gantt chart ({active_day})",

```

```

        "l": f"View surgery list ({active_day})",
        "w": "Weekly summary",
        "d": "Change active day",
        "a": "Add surgery",
        "c": "Cancel a surgery",
        "e": "Inject emergency case",
        "s": "Scenario testing",
        "p": "Pilot testing (standardized benchmarks)",
        "x": "Change MILP parameters",
        "r": "Regenerate week",
        "o": "Export to Excel report",
        "q": "Quit",
    }
    ch = menu_prompt(options)

    if ch == "g":
        print_gantt(room_list, params, title=f"{active_day} - Optimized Sched
        print_kpis(kpis, params, label=active_day)

    elif ch == "l":
        print_surgery_list(room_list, active_day)
        print_kpis(kpis, params, label=active_day)

    elif ch == "w":
        print_weekly_summary(week, optimizer, rooms)

    elif ch == "d":
        active_day = get_day_choice()
        print(clr(f"\n  ✓ Active day → {active_day}", G))

    elif ch == "a":
        week = add_surgery_interactive(week, active_day)

    elif ch == "c":
        week = cancel_surgery_interactive(week, room_list, active_day)

    elif ch == "e":
        week = inject_emergency(week, active_day)
        room_list, kpis = optimizer.optimize(week.get(active_day, []), rooms)
        print_gantt(room_list, params, title=f"{active_day} - Post-Emergency
        print_kpis(kpis, params, label=f"{active_day} (post-emergency)")

    elif ch == "s":
        run_scenarios(week, active_day, optimizer, rooms, kpis, params)

    elif ch == "p":
        pilot_mode_test(week, active_day, optimizer, rooms, params)

```

```

elif ch == "x":
    print(clr("\n — Modify MILP Parameters —", BLD, M))
    try:
        params.alpha = float(input(clr(f" ►  $\alpha$  (idle cost) [{params.alp
        params.beta = float(input(clr(f" ►  $\beta$  (OT cost) [{params.bet
        params.Tmax = int(input(clr(f" ► Tmax (min) [{params.Tmax}
        params.cleanup = int(input(clr(f" ► Ci (cleanup) [{params.clean
        optimizer = MILPOptimizer(params)
        print(clr(f"\n ✓ Parameters updated:  $\alpha$ ={params.alpha}  $\beta$ ={params.
    except:
        print(clr(" ✗ Invalid input - parameters unchanged", R))

elif ch == "r":
    week = generate_week()
    print(clr(" ✓ New week generated and reoptimized", G))

elif ch == "o":
    fname = f"OR_Report_{datetime.now().strftime('%Y%m%d_%H%M')}.xlsx"
    out = export_excel(week, optimizer, rooms, params, f"/mnt/user-data/o
    print(clr(f"\n ✓ Excel report exported: {fname}", G, BLD))
    print(clr(f" Sheets: Weekly Summary, {' '.join(DAYS)}, MILP Param

elif ch == "q":
    print(clr("\n Goodbye. OR Scheduler terminated.\n", GR))
    sys.exit(0)

else:
    print(clr(" ✗ Unknown option", R))

if __name__ == "__main__":
    main()

```